

Chapter 6

Thread Safety

As we have seen in previous chapters, ensuring correctness of concurrent programs is not straightforward. It requires reasoning about all possible program interleavings, which can be a very large (even infinite) set. Furthermore, in practice, this type of reasoning does not always scale to large software systems.

In practice, software engineers reason about the correctness of concurrent programs in a modular way. In Java, engineers reason at the level of classes. In a nutshell, the idea is to design a class such that multiple threads can use it concurrently without producing errors and without having to use synchronization mechanisms outside the class [3].

When objects of a class can be used concurrently, it is common to say that the class is *thread safe*. However, this term is informal and has multiple meanings. In this chapter, we introduce a precise definition of thread safe class based on the theoretical foundations of the Java memory model. Furthermore, we describe the main properties to take into account to ensure that a class is thread safe.

6.1 Thread safe Java classes

To have a precise definition of thread safe class, we re-use the definition of correctly synchronized program of the Java memory model. Recall that a Java program is correctly synchronized if none of its executions contain data races (see the chapter on the Java memory model for the details). Thus, for a Java class, we define thread safety as follows:

Definition 17 (Thread safe class). A class is *thread safe* if no concurrent execution of field accesses and method calls on objects of this class results in data races on the fields of the class.

In what follows, we use the term *class state* to refer to the fields of the class.

6.2 How to determine whether a class is thread safe

Definition 17 is very useful, as it allows software engineers to use thread safe classes concurrently without worrying about synchronization. However, the definition only states the guarantees that a thread safe class must enforce. It does not say how to construct thread safe classes. To ensure that a class is thread safe, the following properties must hold for the class state (i.e., for all the fields in the class):

```

// DO NOT use this class concurrently
class NonThreadSafeMaxSumConcurrentList {
    private List<Integer> list; // Unsafely published, can escape via getList()
    int sum = 0;                // Can escape via direct field access
    private int maxSum;         // Not safely published

    public NonThreadSafeMaxSumConcurrentList(int maxSum) {
        this.list = new ArrayList<Integer>(); // Not safely published
        this.maxSum = maxSum;                // Not safely published
    }

    public void add(Integer i) throws Exception {
        if (sum + i <= maxSum) { // Accesses mutable state without mutual exclusion
            list.add(i);         // Accesses mutable state without mutual exclusion
            sum += i;            // Accesses mutable state without mutual exclusion
        } else {
            throw new Exception("Cannot add "+i+" to the list");
        }
    }

    public int getSum() {
        return sum;             // Accesses mutable state without mutual exclusion
    }

    public int getMaxSum() {
        return maxSum;
    }

    public List<Integer> getList() {
        return list;           // Makes list reference escape
    }
}

```

Listing 6.1: Example class with multiple thread safety issues.

- No escaping
- Safe publication
- Immutability or mutual exclusion

Ensuring thread safety by enforcing the properties above was first discussed by Goetz *et al.* [3]. Furthermore, in [7], the authors formally prove—using the Java memory model—that classes that enforce these properties are thread safe (according to definition 17).

In what follows, we discuss each property separately. To this end, we will use the classes in listings 6.1 and 6.2. Both implement an integer list class where new integers can be added as long as their sum does not go over a given maximum value. Listing 6.1 contains examples of thread safety errors for each of the properties mentioned above. Listing 6.2 shows a correct thread safe version where problems in listing 6.1 have been solved.

6.2.1 Thread safety 1: No escaping

To prevent class state from escaping one must ensure that class state can only be accessed through public methods. This allows developers to control how the fields of the class are accessed, and use appropriate synchronization mechanisms (on the public methods) if needed.

Definition 18 (No escaping). We say that *class state does not escape* if it can only be accessed through public methods.

To ensure that class state does not escape, the following properties must hold in the class:

ESC1 All fields must be declared as `private`. This ensures that the fields of the class cannot be accessed through field access expressions¹ (i.e., using “dot” notation, e.g., `o.f` for an object `o` with field `f`).

ESC2 If the field is of a reference type, then no public method is allowed to return the reference stored in the field. The problem is that a reference returned by a method can later be used to access the object without any synchronization. This is not a problem for primitive types such as `int`, `float`, `boolean` and so on, since for these types only immutable copies of the field values are returned.

Example 12. Consider the non thread safe class in listing 6.1. Fields `sum` and `list` can escape.

In the case of `sum`, the reason is that it can be accessed through field access expressions. The developer of the class cannot control how users of the class will access this field. For instance, given `badlist` of `NonThreadSafeMaxSumConcurrentList`, it is possible that two threads execute `badlist.sum = 42` and `badlist.sum = 21` concurrently and without synchronization. This leads to a data race, as the accesses are conflicting but they are not ordered by happens-before. Consequently, the definition of thread safe class (cf. definition 17) is not satisfied.

In the case of `list`, a reference to the list is leaked through the method `getList()`. We could have two threads that both execute `innerList = badlist.getList(); innerList.add(10);` concurrently, which leads to a data race on field `list` because `innerList` refers to the same object as `list`. For `list`, direct field accesses from outside `NonThreadSafeMaxSumConcurrentList` are not possible, as the field is declared as `private`. Thus, the only way for `list` to escape is through the `getList()` method.

Listing 6.2 solves the problems above by declaring `sum` as `private` and by removing the `getList()` method. The remaining field, `maxSum`, did not have escaping issues, because its type is a primitive type and because it is declared `private`.

6.2.2 Thread safety 2: Safe publication

Definition 19 (Safe publication). Class state is *safely published* if, after initializing an object of the class, the initial values of the fields of the class are visible to any thread accessing the object.

Intuitively, the definition states that if the constructor of an object has been fully executed and we pass a reference to that object to another thread, the receiving thread must be able to read the values set during object initialization (i.e., in variable initializers or in the body of the constructor). In other words, we must ensure that the values written during initialization are visible to all threads accessing the object. In terms of the Java memory model, safe publication

¹<https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html#jls-15.11>

requires that the write access setting the initial value of the fields of the class happens-before any other access (from any thread).

To ensure safe publication, at least one of the following properties must hold in the fields of the class:

- SP1 Initializing the field to the default value. That is, zero for numeric primitive types (`int`, `float`, `long`), false for `boolean`, and null for reference types. This is appropriate only if the default value is a valid initial value for the field, given the intended functionality of the class.
- SP2 Declaring the field as `final`. This is the standard way to ensure visibility in reference type fields. Note that the `final` modifier only makes the reference to the object final; the contents of the referenced object can still be updated. For primitive types, declaring the field `final` works only if the value is never intended to change after initialization.
- SP3 Declaring the field as `static`. This way of ensuring safe publication is appropriate only if the single static field is intended to be shared by all instances of the class.
- SP4 Declaring the field as `volatile`. This way of ensuring safe publication is only adequate if it is meaningful to declare the variable as `volatile`. Otherwise, if the variable does not need to be `volatile`, declaring it as such will entail an unnecessary performance penalty due to the cost of reads and writes on `volatile` fields.

The properties above ensure that the initialization write access happens-before any subsequent access because they are instances of rules in the definition of happens-before (see definition 11 in the Java memory model chapter). Property SP1 correspond to the default init rule. Properties SP2 and SP3 correspond to the final/static field init rule. Finally, property SP4 is an instance of the volatile rule where the initialization write must be the first access on the field.

Example 13. In listing 6.1, the fields `list` and `maxSum` are unsafely published. Since `list` is of a reference type, its default value is `null`. However, the constructor initializes it to an object of class `ArrayList`. The `maxSum` field of type `int` gets initialized to the constructor parameter `i`, but the default value of `int` is zero. As a consequence, none of these initial values are guaranteed to be visible to all threads with access to objects of this class. In terms of the Java memory model, this means that the initial write access and accesses to these fields from other threads are not ordered by happens-before. This can lead to data races, which is not allowed by the definition of thread safe class (cf. definition 17).

The implementation in listing 6.2 fixes the problems above by declaring `list` and `maxSum` to be `final`. For `list`, this is appropriate because only the reference to the `ArrayList` object becomes immutable after initialization. The content of the list can be modified. Also, the field `maxSum` is not intended to change after initialization, as none of the methods of the class perform write accesses on it. As a consequence, it is adequate to declare it as `final`.

The field `sum` is initialized to zero through a variable initializer in both versions of the class. Since zero is the default value for `int`, the field `sum` is safely published in both cases.

6.2.3 Immutability

Definition 20. A field in a class is *immutable* if its initial value is set during object initialization and afterwards it never changes.

Note that this definition concerns use of the field, not whether it is declared `final`. A field may be immutable even though it is not declared `final`, as long as its initial value never changes.

In order to avoid data races on immutable fields, we only need to make sure that they are safely published. Subsequent read accesses do not need to be protected by any synchronization primitives (such as locks). Absence of data races is guaranteed because the only write access on an immutable field occurs during object initialization and safe publication ensures that this access happens-before any subsequent read operation. Furthermore, as it is an immutable field, no other write access can occur, which means that there cannot exist conflicting accesses leading to data races.

Example 14. In listing 6.1, `maxSum` is an example of an immutable field, as it is only written once in the constructor. The only thread safety issue with this field is unsafe publication, as described in example 13. Consequently, the only change in the fixed version of the class (listing 6.2) is to declare the field as `final`. The read access in the method `getMaxSum()` does not need to be protected, as it cannot lead to data races.

6.2.4 Thread safety 3: Mutual exclusion

If any of the methods in the class access mutable class state, then we must ensure mutual exclusion. We introduced the definition of mutual exclusion in chapter 1 (cf. definition 5). For any method, the critical section is defined as the part of the method that accesses mutable class state. A field in a class is *mutable* if there are methods (other than the constructor) that perform write accesses on the field. This is the opposite of our definition of immutability (cf. definition 20).

Note that ensuring mutual exclusion of mutable class state guarantees that no data races can occur from concurrent method calls.

Example 15. In listing 6.1, the fields `list` and `sum` are mutable. For both fields, there are write accesses in the method `add(...)`. Method `getSum()` also accesses `sum`. Since listing 6.1 does not ensure mutual exclusion in `add(...)` or `getSum()`, concurrent executions of these methods have data races.

Listing 6.2 uses Java intrinsic locks (via `synchronized`) to ensure mutual exclusion when executing `add(...)` and `getSum()`. No other methods access mutable state; we removed `getList()` because it allowed `list` to escape. Since `maxSum` is immutable, these modifications guarantee that no concurrent execution of method calls contains data races.

In practice, some software engineers would ensure mutual exclusion only in the `add(...)` method, and not in `getSum()`. The reasoning behind this choice is that reading a stale value of `sum` is not problematic, and using synchronization (e.g., acquiring an intrinsic lock) introduces an unnecessary performance overhead. While this might be a reasonable assumption for a particular application, it does not satisfy our definition of thread safe class (cf. definition 17). If we do not use `synchronized` in `getSum()`, then concurrent executions of `add(...)` and `getSum()` will produce data races on the field `sum`. In the course, we do not allow this type of relaxation. Mutual exclusion must always be ensured when accessing mutable class state, as otherwise our definition of thread safe class (definition 17) is not satisfied.

```

class ThreadSafeMaxSumConcurrentList {
    private final List<Integer> list; // Cannot escape via method calls nor
                                     // direct field access;
                                     // final ensures safe publication
    private int sum = 0; // Cannot escape via direct field access;
                       // safely published due to default initial value
    private final int maxSum; // Cannot escape via direct field access;
                           // final ensures safe publication

    public ThreadSafeMaxSumConcurrentList(int maxSum) {
        this.list = new ArrayList<Integer>();
        this.maxSum = maxSum;
    }

    // synchronized ensures mutual exclusion when accessing mutual state
    // (i.e., list and sum)
    public synchronized void add(Integer i) throws Exception {
        if (sum + i <= maxSum) {
            list.add(i);
            sum += i;
        } else {
            throw new Exception("Cannot add "+i+" to the list");
        }
    }

    // synchronized ensures mutual exclusion when accessing mutable sum
    public synchronized int getSum() {
        return sum;
    }

    // Since maxSum is immutable, no mutual exclusion is required to read
    public int getMaxSum() {
        return maxSum;
    }
}

```

Listing 6.2: Thread safe version of the example class in listing 6.1.